arXiv:2309.09506v2 [cs.CV] 19 Sep 2023

# LAYOUTNUWA: REVEALING THE HIDDEN LAYOUT EXPERTISE OF LARGE LANGUAGE MODELS

**Zecheng Tang**[1,2*] **Chenfei Wu**[2*] **Juntao Li**[1] **Nan Duan**[2†]
[1]Soochow University [2]Microsoft Research Asia
{zctang@stu., ljt}@suda.edu.cn, {chewu,nanduan}@microsoft.com

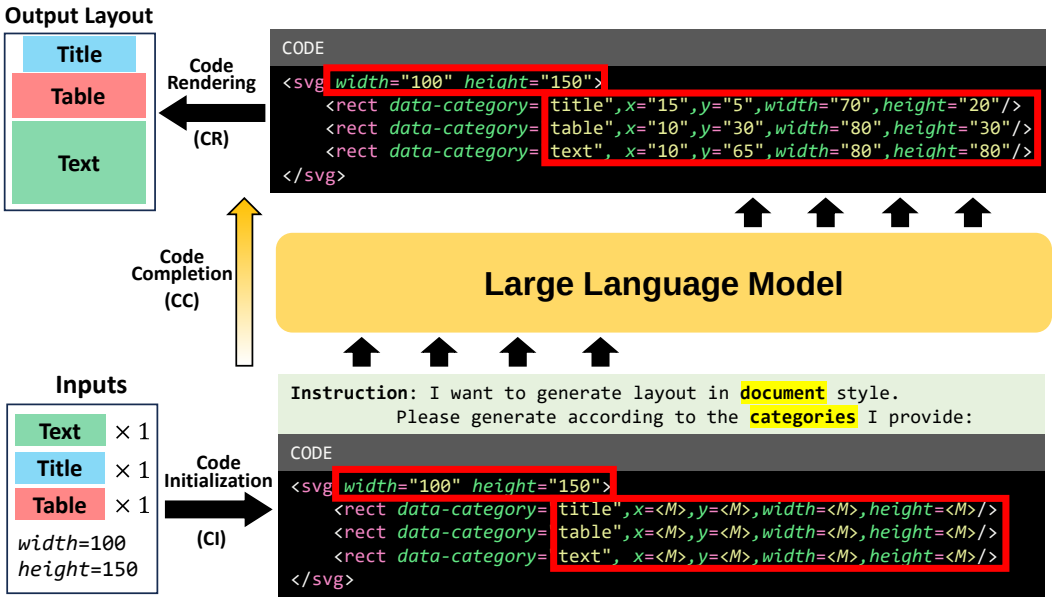



Figure 1: Overview of LayoutNUWA, in which we view layout generation as a code generation task to enhance the semantic information in layouts as well as naturally harness the hidden layout expertise of large language models. In detail, we propose a Code Instruct Tuning (CIT) approach that consists of three modules: 1) the Code Initialization (CI) module quantifies the numerical conditions and initializes them as an HTML code with masks; 2) the Code Completion (CC) module utilizes the knowledge of large language models to complete the masked portions within the HTML code; 3) the Code Rendering (CR) module directly renders the completed code into the final graphic layout.

## ABSTRACT

Graphic layout generation, a growing research field, plays a significant role in user engagement and information perception. Existing methods primarily treat layout generation as a numerical optimization task, focusing on quantitative aspects while overlooking the semantic information of layout, such as the relationship between each layout element. In this paper, we propose LayoutNUWA, the first model that treats layout generation as a code generation task to enhance semantic information and harnesses the hidden layout expertise of large language models (LLMs). More concretely, we develop a Code Instruct Tuning (CIT) approach comprising three interconnected modules: 1) the Code Initialization (CI) module quantifies the numerical conditions and initializes them as HTML code with strategically placed masks; 2) the Code Completion (CC) module employs the formatting knowledge of LLMs to fill in the masked portions within the HTML code; 3) the Code Rendering (CR) module transforms the completed code into the final layout output, ensuring a highly interpretable and transparent layout generation procedure that directly maps code to a visualized layout. We attain significant state-of-the-art performance (even over 50% improvements) on multiple datasets, showcasing the strong capabilities of LayoutNUWA. Our code is available at `https://github.com/ProjectNUWA/LayoutNUWA`.

*Both authors contributed equally to this research. During Zecheng's internship under the mentorship of Chenfei at MSRA.
†Corresponding author.

# 1 INTRODUCTION

Graphic layout, which refers to the organization and positioning of design elements, significantly influences the way users engage with and perceive the presented information (Lee et al., 2020). As a growing research field, layout generation (Li et al., 2019; Yang et al., 2020) aims to create diverse and realistic layouts that streamline the design process and cater to various applications, such as user interfaces (Deka et al., 2017; Jiang et al., 2022), indoor scenes (Di & Yu, 2021; Feng et al., 2023), document layouts (Zheng et al., 2019; Yamaguchi, 2021), presentation slides (Fu et al., 2022), etc.

Current approaches (Jyothi et al., 2019; Li et al., 2019; Arroyo et al., 2021; Zhang et al., 2023a) regard each element in the layout as numerical tuples $(c, x, y, w, h)$, in which $c$ indicates the element category, $x$ and $y$ represent coordinates, $w$ and $h$ correspond to width and height. For example, autoregressive-based methods (Yang et al., 2020; Jiang et al., 2022) view the tuple as a sequence and predict their values sequentially, while diffusion-based methods (Chai et al., 2023; Inoue et al., 2023) consider the tuple as a whole and predict their values through a denoising approach. Despite adopting different generative models, all of these methods fundamentally consider layout generation as a numerical tuple optimization task. However, representing layouts as numerical tuples have its limitations, as it primarily focuses on capturing the quantitative aspects of the layout, such as positions and sizes, while lacking semantic information, e.g., the attribute of each numerical value, which may limit the model's ability to capture more complex and rich layout information.

An insightful question emerges from the limitations of existing methods in layout generation: can we integrate semantic information into the layout generation process to enrich the overall representation and enhance the quality of the generated layouts? Addressing this question brings forth two major benefits: firstly, it bolsters the understanding of relationships among various layout elements, and secondly, it enables us to tap into the semantic capabilities of LLMs (Tang et al., 2023), resulting in more intricate and contextually relevant layouts for a wide range of applications (Jiang et al., 2022). Considering the inherent logical nature of layouts, which involve dependency relationships among layout elements, and the fact that each graphic layout can be represented with a fixed structure sequence, code languages emerge as a promising alternative. Code languages can encompass numerical and semantic information while possessing a strong logical foundation (Chen et al., 2022), which can thus bridge the gap between existing methods and the desired enriched representation.

Based on the above observations, we propose LayoutNUWA, a groundbreaking model that revolutionizes the layout generation task by treating it as a code generation task. Our innovative approach is designed to not only enhance the semantic information within layouts but also seamlessly leverage the expertise of LLMs in the layout generation process. To achieve this, we design a Code Instruct Tuning (CIT) approach comprising three interconnected modules: 1) firstly, the Code Initialization (CI) module quantifies the numerical conditions and initializes them as HTML code with strategically placed masks, paving the way for more meaningful and coherent layouts; 2) secondly, the Code Completion (CC) module employs the formatting knowledge of LLMs to fill in the masked portions within the HTML code, thereby harnessing the power of LLMs to improve the accuracy and consistency of the generated layouts; 3) lastly, the Code Rendering (CR) module transforms the completed code into the final layout output, ensuring a highly interpretable and transparent layout generation procedure that directly maps code to a visualized layout.

Experiments across a variety of conditional layout generation tasks on three datasets, i.e., Rico (Deka et al., 2017), PubLayNet (Zhong et al., 2019) and Magazine (Zheng et al., 2019), highlight the superiority of our method, in which LayoutNUWA can significantly outperform all the baselines and shows comparable results with the task-specific models. Furthermore, LayoutNUWA can achieve at least a 50% improvement in performance compared to the best baseline on the low-resource datasets, e.g., the Magazine dataset. In a nutshell, our contributions can be outlined as follows:

- We introduce LayoutNUWA, the first model that treats the layout generation task as a code generation task, effectively harnessing the hidden layout expertise of LLMs.
- We propose Code Instruct Tuning, which empowers the model to adhere to instructions and enriches the semantic information of layout, resulting in precise and standardized code.
- We attain significant state-of-the-art performance on multiple datasets, showcasing the robust capabilities of LayoutNUWA.

# 2 RELATED WORK

## 2.1 LAYOUT GENERATION

Automatic layout generation, an important task for automatic graphical design for various scenarios such as document layouts (Zheng et al., 2019; Zhong et al., 2019; Yamaguchi, 2021; Fu et al., 2022), posters (Yang et al., 2016; Guo et al., 2021; Li et al., 2023) and user interface (Deka et al., 2017), has been recently extensively researched. Early approaches for layout generation involve embedding design rules into manually-defined energy functions (O'Donovan et al., 2014; O'Donovan et al., 2015), while other methods have explored generative models such as GANs and VAEs for generating numerical graphic and scene layouts, including LayoutGAN (Li et al., 2019), LayoutVAE (Jyothi et al., 2019), LayoutGAN++ (Kikuchi et al., 2021), NDN (Lee et al., 2020) and READ (Patil et al., 2020). Apart from them, transformer-based approaches utilize self-attention mechanisms to learn numerical contextual relationships between elements and achieve layout completion based on partial layout inputs (Yang et al., 2020; Kong et al., 2022; Feng et al., 2023). Recently, with the prevalence of diffusion models, several works also adopted diffusion models to tackle a broader range of conditional layout generation (Chai et al., 2023; Inoue et al., 2023; Zhang et al., 2023a; Hui et al., 2023; Cheng et al., 2023). However, existing methods primarily treat layout generation as a numerical optimization task, focusing on quantitative aspects while overlooking the semantic information of layout, such as the relationship between each layout element. Different from previous works, we convert the layout generation task into the code generation task to directly generate the layout in code language and thus utilize the rich knowledge from LLMs, which can significantly improve the FID by 50% in the Magazine dataset in § 4.2.

## 2.2 INSTRUCTION TUNING

Instruction tuning represents the process of fine-tuning LLMs on the instruction dataset in a supervised fashion, which narrows the gap between the next-word prediction manner of LLMs and the users' objective of having LLMs adhere to human instructions (Zhang et al., 2023c). Early attempts on instruction tuning involve multi-task training with manually-written descriptions about different tasks (Mishra et al., 2021; Wei et al., 2021; Sanh et al., 2021; Xu et al., 2022; Muennighoff et al., 2022; Iyer et al., 2022) or automatically generated instructions (Wang et al., 2022; Gu et al., 2022; Zhang et al., 2023b; Honovich et al., 2022a;b). Apart from controlling the LLMs through input instruction, Nye et al. (2021) show that LLM can handle more complex tasks by generating the intermediate steps and Wei et al. (2022) propose chain-of-thought technique by enriching the instruction with intermediate reasoning step descriptions, which endows LLMs with better performance (Wang et al., 2022; Zelikman et al., 2022; Wu et al., 2023; Xu et al., 2023). However, the instruction tuning methods mentioned above are primarily intended for text generation tasks and not ideal for layout generation tasks, which involve numerical optimization. Thus, we propose a code instruction tuning method that is specially designed for layout generation task. Experiments in § 5.1 indicate that the performance significantly drops if the code instruction tuning is not adopted.

# 3 METHODOLOGY

## 3.1 PROBLEM FORMULATION

The layout generation task aims to generate a well-organized layout $\mathcal{S} = \{s_i\}_{i=1}^{N}$, with $N$ representing the number of elements in the layout. Each element, $s_i = (c_i, x_i, y_i, w_i, h_i)$, consists of the following components: $c_i$ is the category, $x_i, y_i$ indicate the center location, and $w_i, h_i$ represent the width and height, respectively. In this study, we focus on the conditional layout generation task, wherein partial components in $s_i$ are masked with $M$, and the complete layout $S$ should be predicted by model $f_\theta$ conditioned on the remaining components $S_{\setminus M}$:

$$\mathcal{S} = f_\theta(\mathcal{S}_{\setminus M}) \tag{1}$$

Previous works (Jyothi et al., 2019; Yang et al., 2020; Inoue et al., 2023) regard each element $s_i$ as a sequence of numerical values, e.g., (0, 10, 20, 25, 30), and train a model to directly generate these values. However, this approach overlooks the semantic information of the components, thus limiting the model's understanding of the layout semantics. Based on this observation, we propose

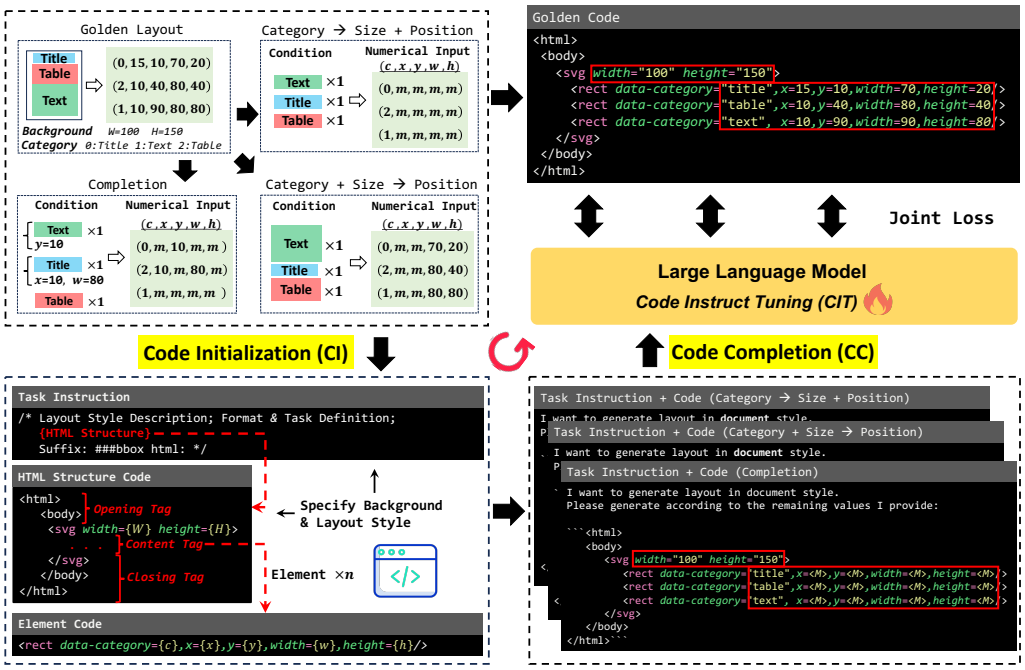



Figure 2: The training process of LayoutNUWA, which converts layout generation task to code generation task and utilizes a code instruct tuning to leverage LLM's capability for layout generation.

a new problem definition, where we convert the input $S_{\setminus M}$ and output $S$ into a code language and view the layout generation task as a code generation task:

$$\mathrm{CODE}(\mathcal{S}) = f_\theta(\mathrm{CODE}(\mathcal{S}_{\setminus M})) \tag{2}$$

Eq. 2 has the following 3 advantages compared with Eq. 1:

- **Semantic Insights**: By converting the numerical values into code language, the model can better capture the semantic relationships between different components of the layout.
- **LLM Utilization**: By using code language, the model can further leverage the knowledge of Large Language Models (LLMs) and thus enhance the quality of the generated layouts.
- **Model Scalability**: The code language has a stronger expressive capability compared to numerical values, which allows the addition of more attributes for layout elements.

### 3.2 Code Instruct Tuning

As shown in Fig. 1, we propose Code Instruct Tuning (CIT) with three modules: (1) *Code Initialization* module converts layout into masked code language with dynamic templates; (2) *Code Completion* module inputs the masked code to LLMs to generate complete code; (3) *Code Rendering* module directly renders code to the final graphic layout. We illustrate these modules below.

#### 3.2.1 Code Initialization

**Element Quantization** We quantify the numerical values of $i$-th element position $\{x_i, y_i\}$ and size $\{w_i, h_i\}$ in the layout with Adaptive Quantization method (Inoue et al., 2023) that applies $k$-Means algorithm (MacQueen et al., 1967) to cluster the position and size information of each element, addressing the highly imbalanced distribution of these values, e.g., elements may overlap or cluster together. Different from the previous works (Chai et al., 2023; Zhang et al., 2023a; Inoue et al., 2023), we use absolute position to represent the coordinates rather than relative positions. This aligns with code language and allows direct rendering of layouts without necessitating coordinate conversion, thereby preventing potential information loss. We maintain precision up to one decimal place and directly convert the clustered results into strings.

**Template Construction** The overview of template construction is shown in Fig. 2. We construct the templates based on the most common web page layout code, HTML, which contains a wealth of information and is easily accessed by LLMs during the pre-training process (Touvron et al., 2023; Rozière et al., 2023). Specifically, in HTML code, each element is described with a tag that provides information about the content or the element structure. Since the elements in the layout are regular squares, we chose the `<rect>` tag as the content tag to describe each element:

```
<rect data-category={c_i} x={x_i} y={y_i} width={w_i} height={h_i}>
```

where $c_i$ is the element category in textual format and $\{x_i, y_i, w_i, h_i\}$ are the quantified position and size of the $i$-th element. Then, to combine all the elements into a unified structure, we used an opening tag and a closing tag to define the boundaries of each layout, which can be written as:

```
<html><body><svg width={W} height={H}> ... </svg></body></html>
```

where $W$ and $H$ are the background width and height of the layout.

In order to facilitate better learning of layout in various domains and tasks and leverage the instruction-following capabilities of LLMs, we design the following prompts:

```
I want to generate layout in {Domain} style. Please generate the
   layout according to the {Task Condition} I provide:
```

where the {`domain`} and the {`Task Condition`} will vary according to different domains and tasks. For instance, for the RICO dataset, we set `Domain` as "mobile UI", and for the layout completion task, we set `Task Condition` as "remaining values". Afterwards, we prepend the task instruction before the layout code.

### 3.2.2 Code Completion

To construct the conditional input of the layout generation task, we utilize the mask tokens of LLMs to represent the masked values $M$ and let the model predict the masked values within the HTML code. Different from previous works (Chai et al., 2023; Zhang et al., 2023a; Inoue et al., 2023) that applied the customized numerical vocabulary, we employ the LLM's token vocabulary directly. By doing so, we can leverage the knowledge of the numerical tokens inherit in the LLMs. Considering that almost all the LLMs follow auto-regressive generation manner and it brings significant limitation to the layout generation task since the model should predict the same layout under different element orders, even if the layout doesn't have a naturally defined order (Yang et al., 2020). Thus, we design a self-consistency strategy that randomly permutes the order of the input elements in the layout within a mini-batch. Meanwhile, in order to adapt LLMs to different conditional layout generation tasks, we have performed multi-task modeling on the same layout, utilizing various conditions and implementing a joint loss for these tasks. Given the permutation times $K$ and task numbers $T$, the joint loss for each layout $\mathcal{S}$ can be written as:

$$L(\mathcal{S} \mid \theta) = \sum_{t=1}^{T} \sum_{j=1}^{N} \sum_{k=1}^{K} L(s_j^{(k)} \backslash M_j^{(t)} \mid \theta), \tag{3}$$

where $\theta$ is the model parameters and $s_j$ denote the $j$-th element in the layout $\mathcal{S}$.

### 3.2.3 Code Rendering

Most existing works require the extra conversion step to render the graphic layouts (Yang et al., 2020; Chai et al., 2023; Zhang et al., 2023a), e.g., converting the relative position to the absolute position, causing the information loss. Different from previous work, LayoutNUWA allows for the immediate rendering as it generate the absolute position directly. Besides, considering the potential output issues such as boundary overflow (Inoue et al., 2023) and format errors, we employ regular expressions to remove mismatched formats and implement clipping operations for elements that exceed the background size.

# 4 EXPERIMENT

## 4.1 EXPERIMENTAL SETTINGS

**Dataset** We evaluate the model performance on three widely used public datasets. RICO (Deka et al., 2017) is a user interface design dataset for mobile applications containing 25 element categories and 66K+ UI layouts. PubLayNet (Zhong et al., 2019) consists of 360K+ layouts for documents with 5 element categories. Magazine (Zheng et al., 2019) is a low-resource magazine layout dataset containing around 4K annotated layouts and 6 element categories. We follow LayoutDM (Inoue et al., 2023) to view the original validation data as the testing set and pre-process all three datasets by discarding the layouts containing more than 25 elements as well as splitting the filtered data into the training and new validation sets by 95% and 5%.

**Evaluation Metrics** We employ four metrics to evaluate the generation results comprehensively, including Frechet Inception Distance (FID), Maximum Interaction over Union (mIoU), Alignment (Align.), and Overlap. Among them, FID compares the distribution of generated and real layouts. Similar to the previous work (Inoue et al., 2023), we utilize an enhanced feature extraction model for layouts (Kikuchi et al., 2021) to compute the FID score. We measure the conditional similarity between generated and real layouts using mIoU, which is done by calculating the maximum IoU between bounding boxes of generated and real layouts with the same type set. Alignment and Overlap scores are calculated following the previous work (Li et al., 2019) to evaluate proper element alignment and overlapping in a generated layout, and it is worth noting that we ignore normal overlaps, e.g., elements on top of the background, and discard the layouts that failed to generate. For reference, we show the evaluation results between the validation set and test set as Real data.

**Tasks and Baselines** We evaluate LayoutNUWA on three conditional layout generation tasks. These include the Category to Size and Position (C $\rightarrow$ S+P) task, the Category and Size to Position (C+S $\rightarrow$ P) task, and the Completion task. More concretely, the C $\rightarrow$ S+P task requires the model to predict the position and size of the element based on its category. For the C+S $\rightarrow$ P task, the model predicts the position of the element based on both its size and category. Finally, in the completion task, the element's size and position values are randomly masked up to 80%, and the model predicts the entire layout using the remaining values. We compare LayoutNUWA with six strong baselines, including LayoutTrans (Yang et al., 2020), BLT (Kong et al., 2022), LayoutGAN++ (Li et al., 2019), MaskGIT (Chang et al., 2022), DiffusionLM (Li et al., 2022) and LayoutDM (Inoue et al., 2023).

**Implementation Details** We implement LayoutNUWA with two 7B LLMs: LLaMA2[1] (L2) (Touvron et al., 2023) and CodeLLaMA[2] (CL) (Rozière et al., 2023). We train LayoutNUWA with two settings: (1) Domain-Specific (DS) setting, where the model is trained on distinct datasets, and (2) Domain-Agnostic (DA) setting, where the model is trained on all three datasets, including RICO, PubLayNet, and Magazine. The default configuration for LayoutNUWA utilizes CodeLLaMA (CL) and Domain-Agnostic (DA), i.e., LayoutNUWA-L2-DS. We set permutation times $K = 10$ and task numbers $T = 3$. For model training, we use DeepSpeed Library (Rajbhandari et al., 2020) to run all experiments on 64 NVIDIA V100 GPUs. We apply Top-$p$ sampling (Holtzman et al., 2019) for inference, where $p = 0.9$ and the temperature is $0.6$, and set the maximum generation length as 512.

## 4.2 QUANTITATIVE EVALUATION

We report the model performance on three datasets: the Magazine dataset in Tab. 1, RICO, and PubLayNet datasets in Tab. 2. For the Magazine dataset, LayoutNUWA demonstrates a remarkable performance by significantly surpassing all baseline measures across all tasks. Moreover, it outperforms the strong baseline LayoutDM by more than 50% when assessed with the FID metric.

The significant improvements in Tab. 1 are due to three aspects: 1) previous approaches generated numerical values, while LayoutNUWA generates code with labels, which greatly benefits the model by utilizing the semantic information of layout attributes such as width, height, position, and category; 2) none of the previous methods used LLMs. However, we have introduced LLMs for the first

[1] https://huggingface.co/meta-llama/Llama-2-7b
[2] https://huggingface.co/codellama/CodeLlama-7b-hf

| Model | Layout Format | LLM | Domain | C → S + P | | C + S → P | | Completion | |
|---|---|---|---|---|---|---|---|---|---|
| | | | | mIOU (↑) | FID (↓) | mIOU (↑) | FID (↓) | mIOU (↑) | FID (↓) |
| LayoutTrans | Numerical | - | Specific | 0.116 | 36.207 | 0.153 | 33.931 | 0.228 | 25.804 |
| BLT | Numerical | - | Specific | 0.087 | 65.372 | 0.126 | 41.089 | 0.103 | 97.142 |
| LayoutGAN++ | Numerical | - | Specific | 0.259 | 16.952 | 0.293 | 11.569 | - | - |
| MaskGIT | Numerical | - | Specific | 0.059 | 140.94 | 0.100 | 78.226 | 0.024 | 152.591 |
| DiffusionLM | Numerical | - | Specific | 0.151 | 32.114 | 0.144 | 24.370 | 0.138 | 33.172 |
| LayoutDM | Numerical | - | Specific | 0.234 | 19.206 | 0.308 | 14.265 | 0.328 | 15.804 |
| LayoutNUWA-L2-DS (ours) | Code | LLaMA2 | Specific | 0.260 | 9.741 | 0.358 | 6.682 | 0.418 | 8.257 |
| LayoutNUWA-L2-DA (ours) | Code | LLaMA2 | Agnostic | 0.293 | 9.632 | 0.394 | 7.238 | 0.413 | 8.734 |
| LayoutNUWA-CL-DS (ours) | Code | CodeLLaMA | Specific | 0.293 | 8.985 | 0.348 | **5.355** | 0.410 | **7.341** |
| LayoutNUWA (ours) | Code | CodeLLaMA | Agnostic | **0.312** | **8.791** | **0.418** | 6.755 | **0.495** | 7.572 |
| **Real Data** | - | - | - | 0.348 | 6.695 | 0.348 | 6.695 | 0.348 | 6.695 |

Table 1: Quantitative comparison on Magazine dataset, where the bold font denotes the best result and underline represents the second-best performance.

| Tasks | Models | RICO | | | | PubLayNet | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | mIoU (↑) | Align. (→) | Overlap (→) | FID (↓) | mIoU (↑) | Align. (→) | Overlap (→) | FID (↓) |
| **Condition C → S + P** | LayoutTrans | 0.219 | 0.014 | 13.012 | 11.237 | 0.271 | 0.016 | 3.229 | 38.910 |
| | BLT | 0.203 | 0.013 | 11.743 | 14.260 | 0.232 | 0.009 | 16.742 | 76.499 |
| | LayoutGAN++ | 0.263 | 0.016 | 3.544 | 6.842 | 0.354 | 0.011 | 1.713 | 10.219 |
| | MaskGIT | 0.267 | 0.001 | 26.865 | 27.470 | 0.320 | 0.004 | 1.857 | 16.898 |
| | DiffusionLM | 0.299 | 0.018 | 17.655 | 31.644 | 0.262 | 0.027 | 3.532 | 20.021 |
| | LayoutDM | 0.275 | 0.010 | 11.938 | 3.576 | 0.310 | 0.010 | 0.024 | 7.915 |
| | LayoutNUWA-L2-DS (ours) | 0.351 | 0.009 | 10.190 | 3.728 | 0.337 | 0.009 | 0.058 | 6.986 |
| | LayoutNUWA-L2-DA (ours) | 0.386 | 0.011 | 10.214 | 3.101 | 0.324 | 0.011 | 0.077 | 6.890 |
| | LayoutNUWA-CL-DS (ours) | 0.377 | 0.009 | 10.263 | 3.706 | 0.376 | 0.008 | **0.053** | 6.715 |
| | LayoutNUWA (ours) | **0.445** | **0.004** | **7.943** | **2.524** | **0.385** | **0.001** | 0.086 | **6.579** |
| **Condition C + S → P** | LayoutTrans | 0.311 | 0.011 | 11.902 | 9.368 | 0.315 | 0.013 | 2.531 | 31.627 |
| | BLT | 0.341 | 0.008 | 13.470 | 4.487 | 0.356 | 0.006 | 5.469 | 8.831 |
| | LayoutGAN++ | 0.349 | 0.011 | 9.628 | 6.219 | 0.346 | 0.008 | 2.746 | 9.936 |
| | MaskGIT | 0.331 | **0.003** | 26.390 | 12.898 | 0.384 | 0.005 | 1.950 | 5.453 |
| | DiffusionLM | 0.278 | 0.020 | 11.884 | 15.931 | 0.324 | 0.014 | 3.990 | 16.407 |
| | LayoutDM | 0.391 | 0.009 | 12.072 | **2.288** | 0.381 | 0.010 | 2.041 | 4.175 |
| | LayoutNUWA-L2-DS (ours) | 0.462 | 0.008 | 10.436 | 3.035 | 0.426 | 0.010 | 1.752 | 4.105 |
| | LayoutNUWA-L2-DA (ours) | 0.464 | 0.007 | 10.117 | 2.973 | 0.464 | 0.009 | 1.984 | 3.993 |
| | LayoutNUWA-CL-DS (ours) | 0.469 | 0.007 | 9.856 | 2.984 | 0.466 | 0.009 | 1.610 | 4.012 |
| | LayoutNUWA (ours) | **0.564** | 0.007 | **7.968** | 2.870 | **0.483** | **0.002** | **0.108** | **3.697** |
| **Completion** | LayoutTrans | 0.561 | 0.008 | 10.080 | 3.733 | 0.439 | 0.012 | 2.053 | 8.689 |
| | BLT† | 0.471 | **0.007** | 53.658 | 121.110 | 0.157 | **0.002** | 109.483 | 155.157 |
| | MaskGIT | 0.537 | 0.024 | 9.242 | 33.463 | 0.349 | 0.011 | 4.768 | 12.013 |
| | DiffusionLM | 0.218 | 0.021 | **8.681** | 22.220 | 0.332 | 0.012 | 4.436 | 16.576 |
| | LayoutDM | 0.580 | 0.009 | 15.676 | 9.224 | 0.377 | 0.011 | 1.891 | 7.570 |
| | LayoutNUWA-L2-DS (ours) | 0.610 | 0.009 | 7.239 | 8.875 | 0.407 | 0.010 | 1.337 | 7.337 |
| | LayoutNUWA-L2-DA (ours) | 0.624 | **0.007** | 10.457 | 8.724 | 0.477 | 0.012 | 1.383 | 7.169 |
| | LayoutNUWA-CL-DS (ours) | **0.641** | **0.007** | 7.529 | 8.734 | 0.473 | 0.012 | 1.311 | 7.233 |
| | LayoutNUWA (ours) | 0.616 | **0.007** | 8.123 | **7.542** | **0.481** | 0.009 | **1.292** | **6.929** |
| **Real Data** | - | 0.438 | 0.004 | 8.706 | 6.25 | 0.691 | 0.001 | 0.039 | 1.85 |

Table 2: Quantitative comparison on the RICO and PubLayNet Datasets. For Align. and Overlap metrics, the closer to the real data, the better performance is (indicated by →).

time, which has resulted in significant performance enhancements, i.e., performance has improved from $19.206$ to $9.741$. Furthermore, when we use CodeLLaMA, which is tuned on code language, the performance improves even further to $8.985$; 3) since different domains require distinct layout formats, early numerical-based methods could only be trained in a domain-specific manner. However, LayoutNUWA is based on code structure, which can be trained in a domain-agnostic manner, allowing for complementary among data from various domains, thus further improving FID to $8.791$.

We have also conducted extensive experiments on two other datasets: RICO and PubLayNet, as shown in Tab. 2. The LayoutNUWA notably surpasses all baseline methods in the majority of tasks. Although it does not achieve the best performance in two specific tasks, it still secures at least the second-highest performance in those instances. This shows the strong generalization of the LayoutNUWA. It is worth mentioning that our model also achieves closer Align. and Overlap scores to the Real Data compared to the baselines. Although previous work has suggested that refinement and discriminator processes can contribute to improving the Align. and Overlap (Inoue et al., 2023; Li et al., 2019) scores, our method attains better results without employing these steps.

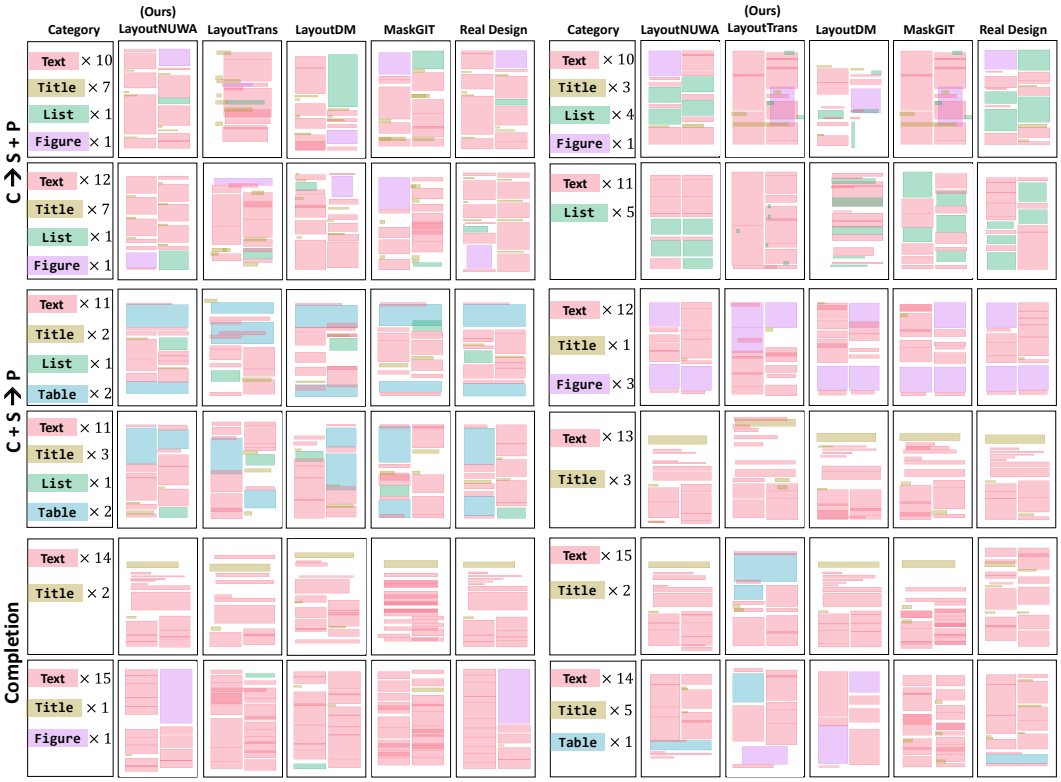



Figure 3: Samples generated by LayoutNUWA on the PubLayNet dataset.

### 4.3 Qualitative Evaluation

We render the generated layout code with the Code Rendering (CR) method, and Fig. 3 shows the sampled rendering results of the PubLayNet dataset. By comparing with other baselines, we can observe that the layouts generated by LayoutNUWA exhibit excellent element alignment, and the proportion of overlap between elements is minimal. Additionally, our results are the most consistent with the Real Design data, i.e., the size and position of the generated element are essentially consistent with the real design, indicating that by treating the layout generation task as a code generation task, LayoutNUWA has successfully learned the distribution of document layouts, thus result in more precise and realistic layouts. More sampled cases can be referred to Fig. 5.

## 5 Ablation Study

We investigate the effectiveness of the CIT tuning method in Sec. 5.1 and compare the impact of different output formats and fine-tuning in Sec. 5.2. More concretely, we set the LayoutNUWA-L2-DS model as the basic setting and conduct the ablation studies on the Magazine dataset.

### 5.1 Effect of Tuning Methods

We progressively reduce the modules in CIT and fine-tune the model using the corresponding constructed data. Specifically, we first exclude the code template and directly convert the element information into an ordered sequence $\boldsymbol{S}$ with a task instruction before it, i.e., the instruction tuning method. Then, we further remove the task instruction and directly fine-tune the model using data from different tasks separately, i.e., the numerical tuning method. As shown in Tab. 3, we can observe that the model performance has declined significantly without the code template, and it can only work in the DS setting since the model can simply generate repetitive and out-of-order results that are inconsistent with the element sequence in the DA setting. Furthermore, the numerical tuning method can only support the DS setting as there is no task instruction for the model to distinguish between different tasks, and the model performance is far inferior compared to those of the CIT as such an approach overlooks the rich semantic information among the elements and can not calibrate the prior code knowledge of LLMs.

| Task | Models | Tuning Method | mIoU (↑) | Align. (→) | Overlap (→) | FID (↓) | Fail (↓) |
|---|---|---|---|---|---|---|---|
| **Condition C → S + P** | LayoutNUWA-L2-DS | CTT | **0.260** | **0.021** | **2.898** | **9.741** | **0.000 %** |
| | w/o template | Instruct Tuning (DS) | 0.124 | 0.049 | 3.221 | 16.324 | 1.020 % |
| | w/o template | Instruct Tuning (DA) | - | - | - | - | 0.000 % |
| | w/o template&instruct | Numerical Tuning | 0.126 | 0.053 | 3.581 | 17.982 | 3.571 % |
| **Condition C + S → P** | LayoutNUWA-L2-DS | CIT | **0.358** | **0.020** | **2.483** | **4.682** | **0.000 %** |
| | w/o template | Instruct Tuning (DS) | 0.182 | 0.021 | 2.673 | 12.432 | 0.000 % |
| | w/o template | Instruct Tuning (DA) | - | - | - | - | 0.000 % |
| | w/o template&instruct | Numerical Tuning | 0.189 | 0.024 | 2.892 | 14.326 | 0.000 % |
| **Completion** | LayoutNUWA-L2-DS | CIT | **0.418** | **0.020** | **2.309** | **7.257** | **0.253 %** |
| | w/o template | Instruct Tuning (DS) | 0.206 | 0.017 | 2.882 | 15.732 | 5.102 % |
| | w/o template | Instruct Tuning (DA) | - | - | - | - | 6.633 % |
| | w/o template&instruct | Numerical Tuning | 0.214 | 0.020 | 3.003 | 16.243 | 6.122 % |
| **Real Data** | - | - | 0.348 | 0.016 | 1.521 | 6.695 | - |

Table 3: Comparison among different tuning methods, where "Fail" is the failure ratio of generation.

| Task | Model | Layout Format | mIoU (↑) | Align. (→) | Overlap (→) | FID (↓) | Fail (↓) |
|---|---|---|---|---|---|---|---|
| **Condition C → S + P** | LayoutNUWA-N | Numerical | 0.000 | 0.000 | 0.867 | - | 78.030 % |
| | LayoutNUWA-L2-DS | Code | **0.260** | **0.021** | **2.898** | **9.741** | **0.000 %** |
| **Condition C + S → P** | LayoutNUWA-N | Numerical | 0.000 | 0.000 | 24.959 | 349.231 | 21.717 % |
| | LayoutNUWA-L2-DS | Code | **0.358** | **0.020** | **2.483** | **4.682** | **0.000 %** |
| **Completion** | LayoutNUWA-N | Numerical | 0.000 | 0.000 | 16.602 | - | 29.293 % |
| | LayoutNUWA-L2-DS | Code | **0.418** | **0.020** | **2.309** | **7.257** | **0.253 %** |
| **Real Data** | - | - | 0.348 | 0.016 | 1.521 | 6.695 | - |

Table 4: Comparison among different output formats.

### 5.2 Effect of Output Format and Finetuning

We compared the effects of the model output in code format and numerical format. For the numerical output format, we designed a Code Infilling task, which involves making the LLM predict only the masked values rather than predicting the entire code sequence. As shown in Tab. 4, we can find that generating in numerical format will increase the failure ratio of model generations, e.g., the model will generate repetitive results, and significantly decrease the model performance. This is because the layout generated by the conditional layout generation task should be logical, while only predicting the masked parts can lead to discrete values that lack logic. Besides, Due to the influence of the autoregressive manner, where the content generated in the next step depends on the previous history, this phenomenon may result in a higher failure probability of model generation when predicting layouts with more masked values. We also conduct a comparison between LayoutNUWA and GPT-4 (Bubeck et al., 2023). Specifically, we allow GPT-4 to perform inference by constructing the input using the CIT method. Tab. 5 shows code instruct tuning for LLM is necessary, as using LLM in a zero-shot manner leads to a high fail rate (100% fail rate of LLaMA2 and around 30% for GPT-4).

| Model | C → S + P | C + S → P | Completion |
|---|---|---|---|
| | Fail (↓) | Fail (↓) | Fail (↓) |
| LLaMA2 (Zero-Shot) | 100.0 % | 100.0 % | 100.0 % |
| CodeLLaMA (Zero-shot) | 100.0 % | 100.0 % | 100.0 % |
| GPT-4 (Zero-Shot) | 34.2 % | 28.8 % | 28.5 % |
| LayoutNUWA | **0.0 %** | **0.0 %** | **0.3 %** |

Table 5: Comparison with LLMs.

## 6 Conclusion

In this paper, we propose LayoutNUWA, a groundbreaking approach that treats layout generation as a code generation task, effectively enriching the semantic information of layouts and leveraging the hidden expertise of LLMs. Extensive experiments on multiple datasets have demonstrated the superiority of our method. This research has the potential to revolutionize the field of layout generation and pave the way for further exploration and development of semantic-aware layout generation approaches in various applications.

## REFERENCES


Diego Martin Arroyo, Janis Postels, and Federico Tombari. Variational transformer networks for layout generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 13642–13652, 2021.

Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023.

Shang Chai, Liansheng Zhuang, and Fengying Yan. Layoutdm: Transformer-based diffusion model for layout generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 18349–18358, 2023.

Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T Freeman. Maskgit: Masked generative image transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11315–11325, 2022.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.

Chin-Yi Cheng, Forrest Huang, Gang Li, and Yang Li. Play: Parametrically conditioned layout generation using latent diffusion. *arXiv preprint arXiv:2301.11529*, 2023.

Biplab Deka, Zifeng Huang, Chad Franzen, Joshua Hibschman, Daniel Afergan, Yang Li, Jeffrey Nichols, and Ranjitha Kumar. Rico: A mobile app dataset for building data-driven design applications. In *Proceedings of the 30th annual ACM symposium on user interface software and technology*, pp. 845–854, 2017.

Xinhan Di and Pengqian Yu. Multi-agent reinforcement learning of 3d furniture layout simulation in indoor graphics scenes. *arXiv preprint arXiv:2102.09137*, 2021.

Weixi Feng, Wanrong Zhu, Tsu-jui Fu, Varun Jampani, Arjun Akula, Xuehai He, Sugato Basu, Xin Eric Wang, and William Yang Wang. Layoutgpt: Compositional visual planning and generation with large language models. *arXiv preprint arXiv:2305.15393*, 2023.

Tsu-Jui Fu, William Yang Wang, Daniel McDuff, and Yale Song. Doc2ppt: automatic presentation slides generation from scientific documents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pp. 634–642, 2022.

Yuxian Gu, Pei Ke, Xiaoyan Zhu, and Minlie Huang. Learning instructions with unlabeled data for zero-shot cross-task generalization. *arXiv preprint arXiv:2210.09175*, 2022.

Shunan Guo, Zhuochen Jin, Fuling Sun, Jingwen Li, Zhaorui Li, Yang Shi, and Nan Cao. Vinci: an intelligent graphic design system for generating advertising posters. In *Proceedings of the 2021 CHI conference on human factors in computing systems*, pp. 1–17, 2021.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*, 2019.

Or Honovich, Thomas Scialom, Omer Levy, and Timo Schick. Unnatural instructions: Tuning language models with (almost) no human labor. *arXiv preprint arXiv:2212.09689*, 2022a.

Or Honovich, Uri Shaham, Samuel R Bowman, and Omer Levy. Instruction induction: From few examples to natural language task descriptions. *arXiv preprint arXiv:2205.10782*, 2022b.

Mude Hui, Zhizheng Zhang, Xiaoyi Zhang, Wenxuan Xie, Yuwang Wang, and Yan Lu. Unifying layout generation with a decoupled diffusion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1942–1951, 2023.

Naoto Inoue, Kotaro Kikuchi, Edgar Simo-Serra, Mayu Otani, and Kota Yamaguchi. Layoutdm: Discrete diffusion model for controllable layout generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10167–10176, 2023.

Srinivasan Iyer, Xi Victoria Lin, Ramakanth Pasunuru, Todor Mihaylov, Dániel Simig, Ping Yu, Kurt Shuster, Tianlu Wang, Qing Liu, Punit Singh Koura, et al. Opt-iml: Scaling language model instruction meta learning through the lens of generalization. *arXiv preprint arXiv:2212.12017*, 2022.

Zhaoyun Jiang, Shizhao Sun, Jihua Zhu, Jian-Guang Lou, and Dongmei Zhang. Coarse-to-fine generative modeling for graphic layouts. In *Proceedings of the AAAI conference on artificial intelligence*, volume 36, pp. 1096–1103, 2022.

Akash Abdu Jyothi, Thibaut Durand, Jiawei He, Leonid Sigal, and Greg Mori. Layoutvae: Stochastic scene layout generation from a label set. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9895–9904, 2019.

Kotaro Kikuchi, Edgar Simo-Serra, Mayu Otani, and Kota Yamaguchi. Constrained graphic layout generation via latent optimization. In *Proceedings of the 29th ACM International Conference on Multimedia*, pp. 88–96, 2021.

Xiang Kong, Lu Jiang, Huiwen Chang, Han Zhang, Yuan Hao, Haifeng Gong, and Irfan Essa. Blt: bidirectional layout transformer for controllable layout generation. In *European Conference on Computer Vision*, pp. 474–490. Springer, 2022.

Hsin-Ying Lee, Lu Jiang, Irfan Essa, Phuong B Le, Haifeng Gong, Ming-Hsuan Yang, and Weilong Yang. Neural design network: Graphic layout generation with constraints. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part III 16*, pp. 491–506. Springer, 2020.

Fengheng Li, An Liu, Wei Feng, Honghe Zhu, Yaoyu Li, Zheng Zhang, Jingjing Lv, Xin Zhu, Junjie Shen, Zhangang Lin, et al. Relation-aware diffusion model for controllable poster layout generation. *arXiv preprint arXiv:2306.09086*, 2023.

Jianan Li, Jimei Yang, Aaron Hertzmann, Jianming Zhang, and Tingfa Xu. Layoutgan: Generating graphic layouts with wireframe discriminators. *arXiv preprint arXiv:1901.06767*, 2019.

Xiang Li, John Thickstun, Ishaan Gulrajani, Percy S Liang, and Tatsunori B Hashimoto. Diffusion-lm improves controllable text generation. *Advances in Neural Information Processing Systems*, 35:4328–4343, 2022.

James MacQueen et al. Some methods for classification and analysis of multivariate observations. In *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*, volume 1, pp. 281–297. Oakland, CA, USA, 1967.

Swaroop Mishra, Daniel Khashabi, Chitta Baral, and Hannaneh Hajishirzi. Cross-task generalization via natural language crowdsourcing instructions. *arXiv preprint arXiv:2104.08773*, 2021.

Niklas Muennighoff, Thomas Wang, Lintang Sutawika, Adam Roberts, Stella Biderman, Teven Le Scao, M Saiful Bari, Sheng Shen, Zheng-Xin Yong, Hailey Schoelkopf, et al. Crosslingual generalization through multitask finetuning. *arXiv preprint arXiv:2211.01786*, 2022.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, et al. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021.

Peter O'Donovan, Aseem Agarwala, and Aaron Hertzmann. Designscape: Design with interactive layout suggestions. In *Proceedings of the 33rd annual ACM conference on human factors in computing systems*, pp. 1221–1224, 2015.

Peter O'Donovan, Aseem Agarwala, and Aaron Hertzmann. Learning layouts for single-pagegraphic designs. *IEEE transactions on visualization and computer graphics*, 20(8):1200–1213, 2014.

Akshay Gadi Patil, Omri Ben-Eliezer, Or Perel, and Hadar Averbuch-Elor. Read: Recursive autoencoders for document layout generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pp. 544–545, 2020.

Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–16. IEEE, 2020.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Victor Sanh, Albert Webson, Colin Raffel, Stephen H Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Teven Le Scao, Arun Raja, et al. Multitask prompted training enables zero-shot task generalization. *arXiv preprint arXiv:2110.08207*, 2021.

Xiaojuan Tang, Zilong Zheng, Jiaqi Li, Fanxu Meng, Song-Chun Zhu, Yitao Liang, and Muhan Zhang. Large language models are in-context semantic reasoners rather than symbolic reasoners. *arXiv preprint arXiv:2305.14825*, 2023.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022.

Lijun Wu, Xu Tan, Di He, Fei Tian, Tao Qin, Jianhuang Lai, and Tie-Yan Liu. Beyond error propagation in neural machine translation: Characteristics of language also matter. *arXiv preprint arXiv:1809.00120*, 2018.

Yang Wu, Yanyan Zhao, Zhongyang Li, Bing Qin, and Kai Xiong. Improving cross-task generalization with step-by-step instructions. *arXiv preprint arXiv:2305.04429*, 2023.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. Wizardlm: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*, 2023.

Hanwei Xu, Yujun Chen, Yulun Du, Nan Shao, Yanggang Wang, Haiyu Li, and Zhilin Yang. Zeroprompt: Scaling prompt-based pretraining to 1,000 tasks improves zero-shot generalization. *arXiv preprint arXiv:2201.06910*, 2022.

Kota Yamaguchi. Canvasvae: Learning to generate vector graphic documents. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 5481–5489, 2021.

Cheng-Fu Yang, Wan-Cyuan Fan, Fu-En Yang, and Yu-Chiang Frank Wang. Layouttransformer: Relation-aware scene layout generation. 2020.

Xuyong Yang, Tao Mei, Ying-Qing Xu, Yong Rui, and Shipeng Li. Automatic generation of visual-textual presentation layout. *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)*, 12(2):1–22, 2016.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35:15476–15488, 2022.

Junyi Zhang, Jiaqi Guo, Shizhao Sun, Jian-Guang Lou, and Dongmei Zhang. Layoutdiffusion: Improving graphic layout generation by discrete diffusion probabilistic models. *arXiv preprint arXiv:2303.11589*, 2023a.

Ruohong Zhang, Yau-Shian Wang, and Yiming Yang. Generation-driven contrastive self-training for zero-shot text classification with instruction-tuned gpt. *arXiv preprint arXiv:2304.11872*, 2023b.

Shengyu Zhang, Linfeng Dong, Xiaoya Li, Sen Zhang, Xiaofei Sun, Shuhe Wang, Jiwei Li, Runyi Hu, Tianwei Zhang, Fei Wu, et al. Instruction tuning for large language models: A survey. *arXiv preprint arXiv:2308.10792*, 2023c.

Xinru Zheng, Xiaotian Qiao, Ying Cao, and Rynson WH Lau. Content-aware generative modeling of graphic design layouts. *ACM Transactions on Graphics (TOG)*, 38(4):1–15, 2019.

Xu Zhong, Jianbin Tang, and Antonio Jimeno Yepes. Publaynet: largest dataset ever for document layout analysis. In *2019 International Conference on Document Analysis and Recognition (ICDAR)*, pp. 1015–1022. IEEE, 2019.

## A LIMITATIONS

Since LayoutNUWA employs the autoregressive (AR) LLMs as the backbone, our method naturally inherits the shortcomings of the AR models:

- The generation speed is slower than the non-autoregressive models (Chang et al., 2022).
- It suffers from the error propagation problem (Wu et al., 2018) especially when training is insufficient, where the content generated later in the sequence may be negatively affected by the errors in the content generated earlier.

In our future work, we will address these challenges and make improvements to generate better graphic layouts.

## B COMPARISON WITH GPT-4

We utilize the GPT-4 model with the commercial API and strictly follow the usage policy [3]. We report the detailed performance of the GPT-4 model in Tab. 6 and show several rendered graphic layouts in Fig. 4. We can observe that the content generated by GPT-4 in the zero-shot setting primarily follows the layout design rule, which further confirms the potential capability of LLMs in generating layouts when guided by the CIT approach. However, when compared to LayoutNUWA, there are several issues with the results generated by GPT-4: 1) the distribution of elements is uneven, with elements tending to be concentrated in certain areas, such as the left side of the canvas; 2) the element sizes are inconsistent, for instance, in some graphic layouts, there might be one or two large elements, which results in the high scores of the mIOU and Overlap metrics for some tasks; 3) there is a significant discrepancy between the data distribution of generated content and the real data.

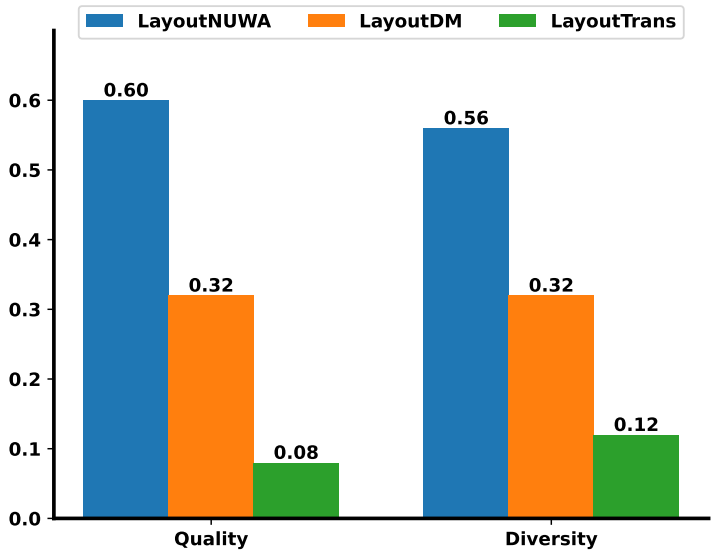
| Task | Model | mIOU (↓) | Align. (→) | Overlap (→) | FID (↓) | Fail (↓) |
|---|---|---|---|---|---|---|
| **Condition C → S + P** | GPT-4 (Zero-Shot) | **0.264** | 0.006 | 0.165 | - | 34.184 % |
| | LayoutNUWA-L2-DS | 0.260 | **0.021** | **2.898** | **9.741** | **0.000 %** |
| **Condition C + S → P** | GPT-4 (Zero-Shot) | 0.330 | 0.011 | **1.149** | - | 28.788 % |
| | LayoutNUWA-L2-DS | **0.358** | **0.020** | 2.483 | **4.682** | **0.000 %** |
| **Completion** | GPT-4 (Zero-Shot) | 0.362 | 0.044 | 0.728 | - | 28.535 % |
| | LayoutNUWA-L2-DS | **0.418** | **0.020** | **2.309** | **7.257** | **0.253 %** |
| **Real Data** | - | 0.348 | 0.016 | 1.521 | 6.695 | - |

Table 6: Detailed performance of GPT4 on the Magazine dataset. It is worth noting that due to the significant difference between the results generated by GPT-4 and the real data, the FID score cannot be calculated.

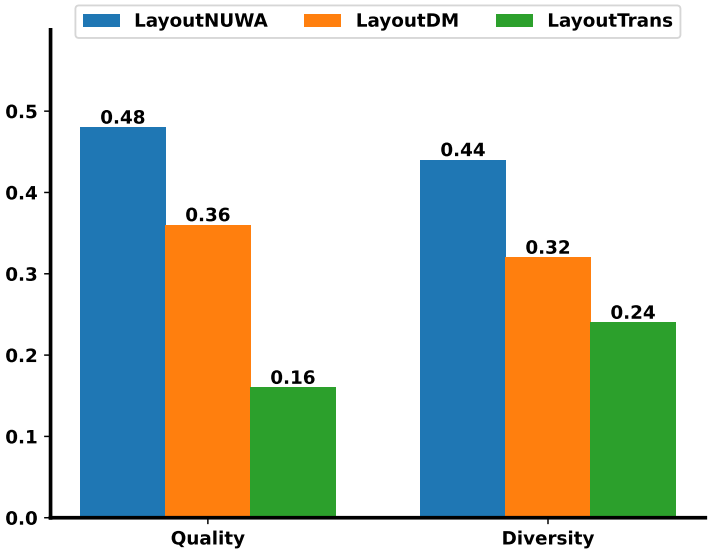
## C HUMAN EVALUATION

We conduct the human evaluation for the model performance on the RICO and PubLayNet datasets. Specifically, We compare LayoutNUWA with two other strong baselines, including LayoutDM (Inoue et al., 2023) and LayoutTransformer (Yang et al., 2020), and randomly sample 25 graphic layouts generated from each model. We invite the annotators to choose which model performs better according to two evaluation settings: 1) *quality evaluation* based on the detail depiction, overlapping degree, and layout rationality in each layout; 2) *diversity evaluation* based on the diversity of the element arrangement in each layout. We hire 10 annotators to give their preferences, and the results are shown in Fig. 4(a) and Fig. 4(b). We can observe that layoutNUWA significantly outperforms the other two strong baselines, i.e., LayoutDM and LayoutTransformer, in terms of both generation quality and generation diversity. More generated cases can be referred to Fig. 4 (Magazine dataset) and Fig. 5 (RICO and PubLayNet datasets).

[3] `https://openai.com/policies/terms-of-use`




(a) Human evaluation on the RICO dataset.




(b) Human evaluation on the PubLayNet dataset.

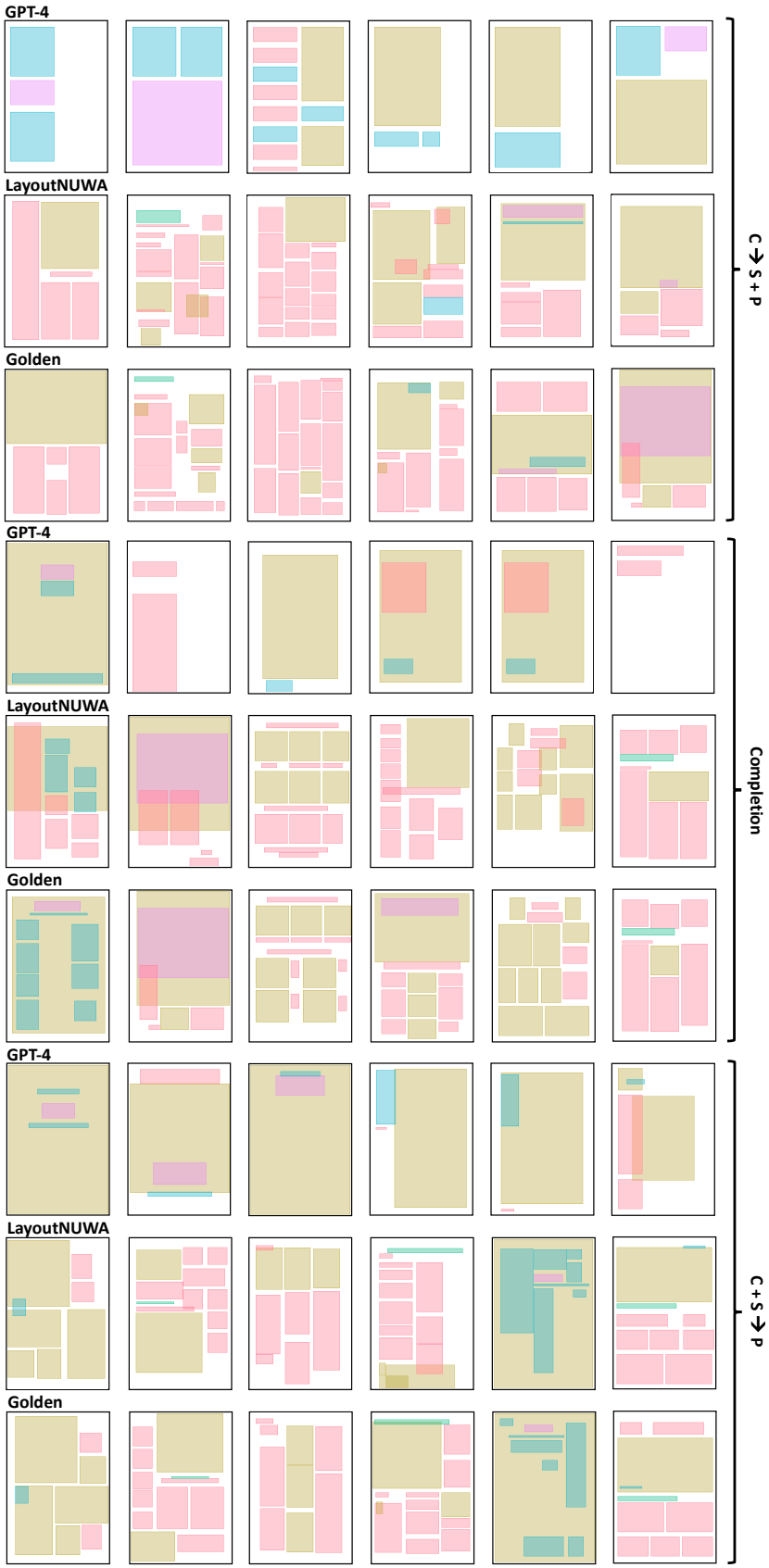



Figure 4: Comparison of rendered graphic layouts between GPT4 and LayoutNUWA on the Magazine dataset.

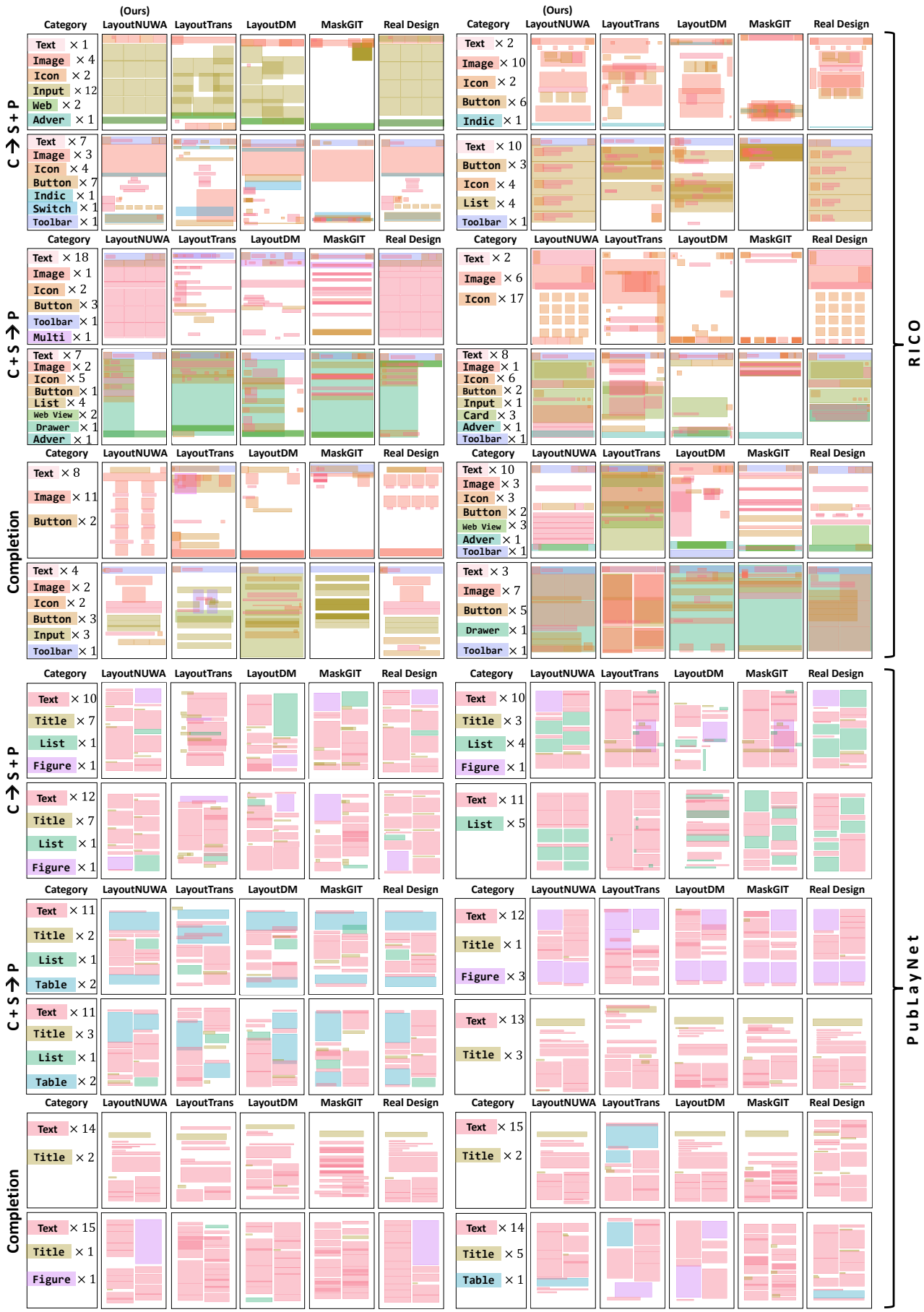



Figure 5: Samples generated by LayoutNUWA on the RICO and PubLayNet dataset.